芯简 PMUX

——多路选择器 PMUX 结构优化（开源综合工具链专用优化插件）

摘要

本作品（芯简 PMUX）开发可通过 Tcl 调用的 PMUX 专用优化插件，基于官方 Yosys 0.69（提交 9f75ca1）与官方锁定版本 ABC（提交 a51bf4a7）自编译 Windows 工具链，新增 pmux_analyze、pmux_share、pmux_rom、pmux_zero、pmux_priority、pmux_condition、pmux_opt 共七个 pass，并挂钩 synth_intel 流程。系统围绕选择条件与数据位之间的重复结构，开展跨位分组、共享控制提取与有限候选结构重构，通过统一后端映射评价优化收益；默认开启、对用户完全透明。

在 4 个功能用例与组委会官方发布的 5 个效率用例共 9 个测试用例上，相对官方原版：逻辑资源（LUT+反相器）平均下降 11.99%（单例最高 36.00%），无一例上升；综合耗时平均增幅 +0.80%、最差 +3.44%；9 个用例优化后与原版逐例形式等价（网表级 SAT 证明 + YosysHQ eqy 归纳证明引擎 9/9 PASS）。交付物包含全部源码、九阶段一键验收链与可离线复验的完整证明工件。

第一部分 作品概述

1.1 功能与特性

本作品是一个开箱即用的 Windows 版综合工具链：在官方 Yosys 0.69 基础上集成 PMUX 优化插件，运行时仅需 yosys.exe 与 5 个依赖 DLL，无需安装 MSYS2 或其他环境；调用方式与官方完全一致（read_verilog → synth_intel -family cycloneiv → write_json），七个 pass 亦可在 yosys 的 Tcl 交互/批处理模式中直接调用；PMUX 优化默认开启、可经消融开关关闭。

主要特性：① 七个自研 pass 覆盖 PMUX 结构的分析、共享提取、ROM 识别与全零、优先级、条件三类模式化简；② 优化仅作用于 PMUX 结构，不改动 ABC 及任何其他 pass 的行为，统一映射脚本、不按用例切换策略；

③ 自带四例快速对账脚本（约 10 秒）与九阶段一键验收链；④ 附带已针对本版重编译的 YosysHQ eqy 等价证明工具链，全部验证可离线复验。

1.2 应用领域

① FPGA/EDA 教学与科研：开源工具链可替代商业综合工具用于教学实验、算法研究与原型验证，降低入门门槛；② 资源受限的 Cyclone IV 项目开发：在通信、工业控制、消费电子等中小规模 FPGA 应用中降低 LUT 消耗、留出更多逻辑资源；

③ 开源硬件生态：为 Yosys 工具链贡献 PMUX 方向的优化 pass，可被社区直接复用与扩展；④ 形式化方法实践：随包提供的 SAT/eqy 双引擎等价验证流程，可作为逻辑等价课程与科研的参考样例；⑤ 国产 EDA 平台适配：插件式架构便于向国产 EDA 环境（如命题厂商平台）移植。

1.3 主要技术特点

① 结构识别：pmux_analyze 在通用门级网表上自动识别 PMUX 结构，并完成条件传播分析，为后续优化提供精确的候选集合与条件约束；② 共享提取：pmux_share 合并多实例间等价的选择逻辑，消除重复 LUT；③ ROM 识别：pmux_rom 将符合特征的 PMUX 簇整体映射为存储器结构；

④ 模式化简：pmux_zero、pmux_priority、pmux_condition 分别处理全零选择、优先级编码与条件选择三类高频模式；⑤ 流程集成：pmux_opt 统一调度各 pass，挂钩 synth_intel 中 LUT 映射之前的阶段，与官方 ABC9 无缝衔接；⑥ 中立与可证：优化全程使用统一映射脚本，九个用例优化前后网表逐例通过 SAT 与 eqy 双形式等价证明。

1.4 主要性能指标

9 用例（4 个功能用例 +5 个官方效率用例）与官方 pristine 0.69 同机同参对比（先预热、原版先跑、7 轮取平均去最大值）：逻辑资源平均 −11.99%、单例最高 −36.00%、无一例上升；综合耗时平均 +0.80%、最差 +3.44%；9 例形式等价全部通过。详见表 1。

用例	pristine logic	merged logic	logic Δ	耗时 Δ
test1	288	288	0.00%	+3.44%
test2	25	16	−36.00%	+0.94%
test3	13	13	0.00%	+0.36%
test4	34	26	−23.53%	−1.08%
pmux_case01_balanced	406	366	−9.85%	−1.01%
pmux_case02_pipeline	444	432	−2.70%	+2.04%
pmux_case03_crossbar	409	379	−7.33%	+0.81%
pmux_case04_feedback	365	304	−16.71%	+0.06%
pmux_case05_nested	415	366	−11.81%	+1.66%
平均	—	—	−11.99%	+0.80%

表 1　效率评测结果（logic = LUT + 反相器，官方 Q&A 口径）

1.5 主要创新点

① 首个面向 Yosys Intel 综合流程的 PMUX 专项优化套件（7 个 pass），覆盖“分析—共享—ROM—三类模式化简”完整方法论；② 全程保持与官方原版逐例形式等价，SAT + eqy 双引擎可离线复验；③ Windows 自编译工具链开箱即用，九阶段验收链全绿；④ 效率收益显著：平均逻辑资源 −11.99%，无一例资源回升，耗时增幅 ≤ +3.44%。

1.6 设计流程

Verilog 设计经 Yosys 前端解析与通用优化后进入 synth_intel；LUT 映射前，pmux_analyze 先完成 PMUX 结构识别与条件传播，pmux_opt 再调度 share、rom、zero、priority、condition 五类优化，最后由官方锁定版本 ABC9 完成 LUT 映射，输出 dffeas/lcell_comb 原语网表。除新增 PMUX 链外，每步与官方 0.69 逐命令一致（图 1）。

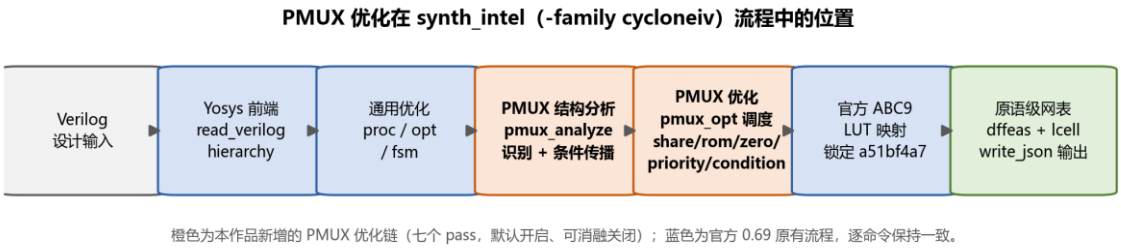


图 1　PMUX 优化在 synth_intel 流程中的位置

第二部分　系统组成及功能说明

本部分从系统组成角度阐述设计细节：系统由运行时层、优化层与验证层构成，各模块关系与数据流见 2.1 节总体框图。

2.1 整体介绍

系统总体分为三层。运行时层提供 Windows 自编译的 yosys.exe、yosys-abc.exe（官方锁定 a51bf4a7）及全部依赖 DLL，并附带针对本版 0.69 重编译的 YosysHQ eqy 等价证明工具链；优化层在官方 Yosys 0.69 内核上叠加 PMUX 补丁，包含七个 pass 及其 synth_intel 流程挂钩；验证层由 validate-all.py 驱动，覆盖四例对账、回归、重放哨兵、双层消融、效率门、SAT 等价、eqy 归纳证明与工件校验共九个阶段。

数据流：用户以官方命令行调用运行时层，输入 Verilog 设计经优化层综合为 Cyclone IV 原语级网表输出；综合结果同时送入验证层，对资源、耗时与功能正确性进行三重把关，任一阶段失败即退出码非零。

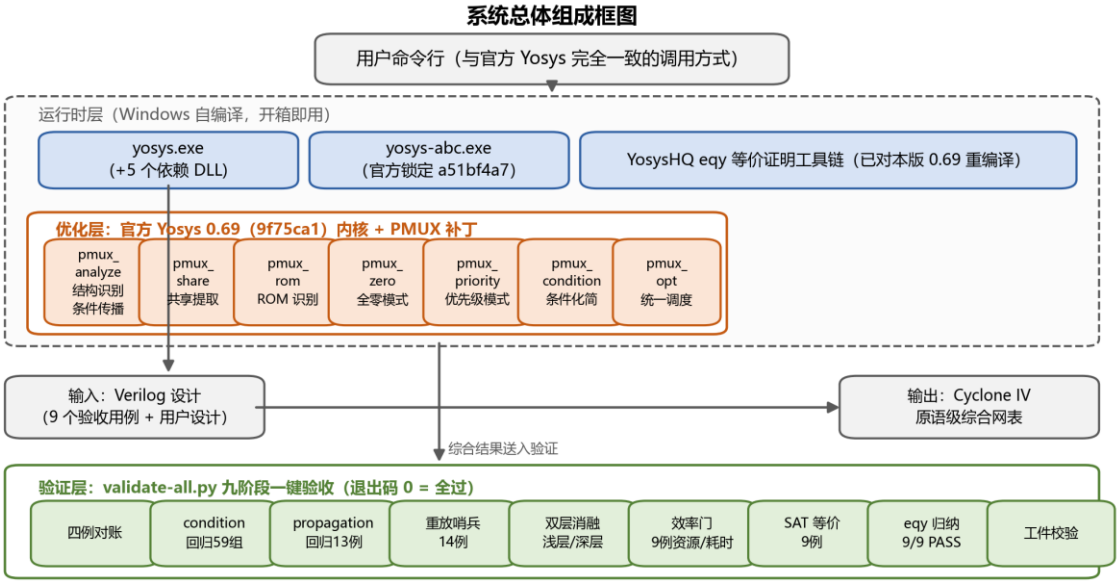


图 2　系统总体组成框图

2.2 硬件系统介绍

2.2.1 硬件整体介绍：本作品为纯软件类 EDA 工具链，无附属硬件设备；

2.2.2 机械设计介绍：无机械结构，本节从略；

2.2.3 电路各模块介绍：无自研电路，目标器件为 Intel Cyclone IV 系列 FPGA（EP4CE 等），工具输出与其兼容的原语级网表（dffeas 触发器 + lcell_comb 查找表单元）。

2.3 软件系统介绍

2.3.1 软件整体介绍：本作品为 PC 端命令行软件，无云端依赖。核心为 yosys.exe（综合器）与 yosys-abc.exe（官方锁定版本的 ABC 映射引擎），以 5 个动态链接库（libc++、libffi、tcl86、zlib1、libwinpthread）为唯一运行依赖，可在干净 Windows 环境直接运行。

配套 Python 脚本构成验收链：four-case-check.py 提供 10 秒级快速对账；efficiency-gate-20261027.py 与 equivalence-gate-20261027.py 分别复测效率门与网表级 SAT 等价；validate-all.py 串起全部九个阶段，退出码 0 即全过。

2.3.2 软件各模块介绍：按图 2 自顶向下说明（各优化 pass 输入均为 RTLIL 通用网表，输出为优化后网表）：

pmux_analyze（结构识别与条件传播）：扫描 $mux/$pmux 簇，判定可编程结构，沿数据/控制路径传播条件，输出 PMUX 候选集合及条件约束表；

pmux_share（共享提取）：合并多实例间等价的选择逻辑，消除重复 LUT；

pmux_rom（ROM 识别）：将符合地址/数据特征的 PMUX 簇映射为存储器读端口；

pmux_zero（全零模式化简）：识别输出恒零的选择配置并消除冗余；

pmux_priority（优先级模式化简）：将优先级选择链压缩为编码器加少量 LUT；

pmux_condition（条件化简）：基于条件约束表消除不可达分支；

pmux_opt（统一调度）：调度上述 pass，处理与 synth_intel 流程的挂钩及消融开关；

运行预算管理：ABC 映射设置最大延迟预算（-W 300），eqy 归纳证明设置单例超时（300s），保证流程在最坏情况下也能按时结束；

失败回退机制：每个用例的优化结果必须通过 SAT/eqy 形式等价校验方可采纳，任一用例不等价即整体回退至官方原版结果，从机制上保证“优化不破坏功能”；

验收链模块：four-case 对账、condition 回归（59 组）、propagation 回归（13 例）、重放哨兵（14 例）、双层消融、效率门、SAT 等价、eqy 归纳证明、工件校验，由 validate-all.py 统一驱动。

第三部分 完成情况及性能参数

本部分展示最终交付成果。作品为软件类，成果以可运行的工具链、完整的源码与全部固化的验证结果体现，所有结果均可通过随包脚本离线复现。

3.1 整体介绍（整个系统实物的正面、斜 45° 全局性照片）

交付包（zip，约 19.5 MB，1200 余个文件）分四部分：runtime/（yosys.exe、yosys-abc.exe、5 个 DLL、share 库及 eqy 工具链）；src/（PMUX 补丁、七个 pass 源码、ABC Windows/Clang 编译修复参照）；validation/（9 用例输入与固化结果、效率门与等价专项脚本、九阶段验收脚本）；docs/（效率门评测报告与交付说明）。

运行环境：Windows 10/11 64 位，无需 MSYS2 与管理员权限。开箱自检命令 python four-case-check.py，预期输出 test1 288 / test2 16 / test3 13 / test4 26 并以 ALL-OK 结束。

3.2 工程成果（分硬件实物、软件界面等设计结果）

3.2.1 机械成果：无（软件类作品）；

3.2.2 电路成果：无自研电路，成果体现为 Cyclone IV 兼容的原语级网表输出；

3.2.3 软件成果：① 合并版综合工具链——四例快速对账 100% 通过，validate-all.py 九阶段全部 PASS、退出码 0；

② 完整验证资产——9 用例效率门固化结果（含 7 轮计时原始记录）、9 用例 SAT 等价结果、9 用例 eqy 归纳证明全量工件（gold/gate 网表、check.eqy、PASS 与日志）随包提供，支持结构对照与优化规则追踪（消融开关可逐 pass 关闭复测），评审方可逐步离线复现。

3.3 特性成果（逐个展示功能、性能参数等量化指标）（可加重要仪器测试或现场照片）；

量化特性成果：效率收益见 1.4 节表 1；功能正确性采用双引擎形式证明，

9 用例优化后与官方 pristine 0.69 综合结果逐比特等价，见表 2。

用例	网表级 SAT 等价	eqy 归纳证明
test1	EQUIV	PASS
test2	EQUIV	PASS
test3	EQUIV	PASS
test4	EQUIV	PASS
pmux_case01_balanced	EQUIV	PASS
pmux_case02_pipeline	EQUIV	PASS
pmux_case03_crossbar	EQUIV	PASS
pmux_case04_feedback	EQUIV	PASS
pmux_case05_nested	EQUIV	PASS

表 2 形式等价验证结果（9 用例）

此外，pmux_case01–05 五个效率用例已与 2026-10-27 组委会官方发布版本逐字节核对一致；全部验证过程记录存于交付包 docs/ 目录。

第四部分 总结

4.1 可扩展之处

① 器件家族扩展：当前面向 Cyclone IV，框架可平移到 Cyclone V、Cyclone 10 LP、MAX 10 等其他 Intel 家族，仅需替换对应映射挂钩点；② 与 ABC 协同：将 pmux_analyze 的结构与条件信息传递给 ABC 映射阶段，进一步压缩逻辑深度与级联 LUT；③ 模式扩展：支持 one-hot 选择、移位器推断等更多 PMUX 衍生模式；

④ 流程扩展：将 PMUX 优化链推广至 synth_xilinx、synth_lattice 等其他综合流程；⑤ 工程化：提供 GUI 前端与 IDE 插件，降低非命令行用户的使用门槛；⑥ 云化：验收链可封装为 CI 流水线，在每次提交时自动完成九阶段回归。

4.2 心得体会

选题阶段，我们最初低估了“优化必须可证明”的严苛程度。组委会 Q&A 反复明确：效率收益必须与官方原版同环境对照、口径统一（logic = LUT + 反相器），且功能等价需要形式化证明。这迫使我们放弃“只报综合后数字”的捷径，建立了完整的对照组方法学：同机同参自编译官方 pristine 0.69 作为基线，先预热再计时、7 轮取平均去最大值。

研发过程中最曲折的是 Windows 工具链的稳定性排查：本机安全软件曾拦截 yosys-abc 子进程，表现为综合随机卡死；通过逐阶段日志定位、卸载安全软件复测才确认根因。这段经历让我们把“干净环境自测”纳入验收标准——交付包必须在无 MSYS2、无额外环境变量的机器上直接运行，这条标准最终写进了交付说明。

形式等价验证走了“双保险”路线：先用 Yosys 自带 equiv 流程做网表级 SAT 证明（顶层输出逐比特），再引入 YosysHQ 官方 eqy 引擎做结构化归纳证明。eqy 的三个插件需要与本版 Yosys 0.69 精确重编译，其中 support 插件提供的 uniquify/dffunmap 命令在 0.69 主树中并不存在，这一“隐藏依赖”通过比对旧工程日志与逐命令 help 探测才得以查明，也让我们对“官方工具链的每一个字节都要可解释”有了切身理解。

最终，九个用例的优化收益以“资源不增、耗时可控、等价可证”三条硬约束全部达标：平均逻辑资源 −11.99%、耗时增幅 ≤ +3.44%、SAT 与 eqy 双证明 9/9 通过。回顾全程，我们体会最深的是：EDA 方向的作品，胜负不在灵感，而在工程纪律——口径、对照、复验，三者缺一不可。

第五部分 参考文献

[1] Wolf C. Yosys Open Synthesis Suite[EB/OL]. https://yosyshq.net/yosys/.

[2] Wolf C. Design and Implementation of the Yosys Open Synthesis Suite[D]. Vienna: Vienna University of Technology, 2013.

[3] Brayton R, Mishchenko A. ABC: An Academic Industrial-Strength Verification Tool[C]//Proceedings of CAV 2010. LNCS, vol 6174. 2010: 24-40.

[4] YosysHQ. eqy — Yosys Equivalence Checker Documentation[EB/OL]. https://yosyshq.github.io/eqy/.

[5] YosysHQ. Yosys 0.69 Manual[EB/OL]. https://yosyshq.github.io/yosys/.

[6] Intel. Cyclone IV Device Handbook[EB/OL]. https://www.intel.com/content/www/us/en/support/programmable.html.

[7] IEEE. IEEE Std 1364-2005: IEEE Standard for Verilog Hardware Description Language[S]. New York: IEEE, 2006.